哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY
1920-2020

Algorithm:
Survival of the Fittest Anisotropic Mutations

Oleg Kuzenkov

Lobachevsky State University of Nizhny Novgorod, RUSSIA

2025

SoFA – Survival of the Fittest Anisotropic Mutations

The algorithm SoFAM solves the following problem:

to find the point of greatest value of a continuous positive function F(z) defined in the hyperparallelepiped Π.

Pre-set parameters and control functions

The algorithm uses the following pre-set parameters and control functions:

- ς_k is **the control function** that regulates **the mutation variance**; a positive numerical sequence that monotonically decreases to zero with an unlimited increase in the step number k; $\varsigma_k \to 0$ as $k \to \infty$
- $\varphi(k)$ is **control function** that regulates **the agent selection** for the next mutation; monotonously depends on the step k, satisfying the condition $\varphi(k) \to \infty$ as $k \to \infty$.
- *G* is the **threshold of the gene mutation type** (strong or weak mutation), a constant that takes values from the interval (0, 1);
- *H* is the parameter of the **mathematical expectation shift**, a constant that takes values from the interval (0, 1);
- *N* is the **time gap between strong and weak mutations**; a selected natural number comparable in magnitude with the maximal number of steps *K*.

Pseudocode of the optimisation algorithm. Algorithm 1. SoFAM.

Require: n, $\Pi \leftarrow \{[a^L_i ; a^R_i], i=1,n\}$, m, Q, N, γ, number_of_steps, Objective_Fun;

population ← Generator(n,Π,m);

population_F ← Calculator_F(population, Objective_Fun);

NP=m

for k:=m+1 → number_of_steps **do**

(reference_point, reference_index) ← Selector(k, population, population_F, NP);

(shift_point 1,shift_index1) ← Selector(k, population, population_F, NP);

(shift point_2,shift_index2) ← Selector(k, population, population_F, NP);

max_index←Find_Index_of_Max(population_F);

max_population ← population[max_index];

mutation ← Mutation_Operator(k, Π, Q, N, reference_point, shift_point_1, shift_point_2, max_population);

NP:=NP+1

populaton[NP] ←mutaition

population_F[NP] ← Objective_Fun(NP)

SumF:=0;

for l := 1 → NP **do**

SumF :=SumF +(popilation_F[l])$^{\phi(k)}$;

end for

pi = 1;

for j := 1 → NP **do**

if ((popilation_F[j])$^{\phi(k)}$/SumF ≥ γ **then**

population[pi] ←populatin[j];

population_F[pi] ←populatin_F[j];

pi := pi +1;

end if

end for

NP:=pi-1

end for

max index ←Find Index of Max(population F);

return population[max index];

Algorithm 2. Generator

Require: *n,Π,m;*

for *i := 1 → m* **do**

  for *j := 1 → n* **do**

    current point[j] ← rand(a^L_i,a^R_i);

  end for

  population[i] ← current point;

end for

return *population*;

Algorithm 3. Calculator_F

Require: *population, Objective_Fun;*

for *i := 1 → m* **do**

    population_F[i] ← Objective_Fun(population[i]);

end for

return *population_F;*

Algorithm 4. Selector

```
Require: k, population, population_F, NP;
for i := 1 → NP do
    probability[i] ← (population_F[i])^φ(k);
end for
max ←probability[1];
for i := 1 → NP do
    if probability[i] > max then
    max ←probability[i];
    end if
end for
sum ←0;
for i := 1 → NP do
    probability[i] ← probability[i]/max;
    sum ←sum+probability[i];
end for
for i := 1 → NP do
    probability[i] ← probability[i]/sum;
end for
probability interval[1] ← 0;
for i := 2 → NP+1 do
    probability interval[i] ← probability interval[i − 1] +probability[i − 1];
end for
r ←rand(probability interval[1],probability interval[m +1]);
index ← 1;
for i := 2 → NP+1 do
    if probability interval[i] ≥ r then index ← i−1;
    break
    end if
end for
return (population[index],index)
```

Algorithm 5. Mutation Operator

Require: *(k,Π,Q,N, reference_point, shift_point 1, shift_point 2,max_population)*

for *i := 1 → m* **do**

if *rand(0,1) < Q* **then**

centre_point ← reference_point[i] +H(k)(max_population[i] − Reference_point[i])+H(k)(shift_point 1[i] − shift_point 2[i]);

B ←arctan((aR i–centre_point/ς(k)) −arctan((a^L_i *−centre_point/ς(k));*

mutated gene ← reference_point[i] +ς(k) · tan(rand(0,1)/B+ arctan((aL i −center point)/ς(k));

else

centre_point ← reference_point[i];

B ←arctan((a^R_i *−centre point)/ς(k+N)) −arctan((*a^L_i *− centre_point)/ς(k + N));*

mutated gene ← reference_point[i]+ς(k)+ Ntan((rand(0,1)/B −arctan((a^L_i *−centre_point)/ς(k + N)));*

end if

if *mutation_gene >* a^R_i **then**

mutation_gene ← a^R_i *;*

end if

if *mutation_gene <* a^L_i **then**

mutation_gene ← a^L_i *;*

end if

mutation[i] ← mutation_gene;

end for

return mutation;

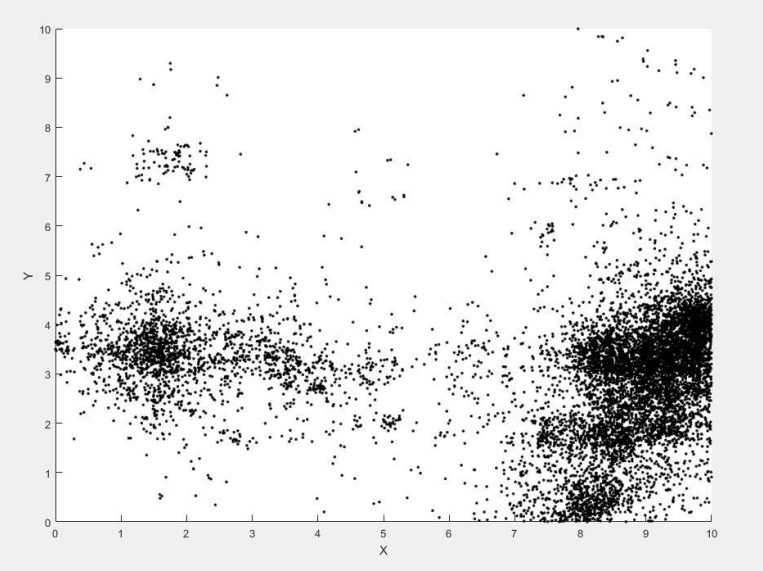
Convergence of algorithm. Theorem 1

Let the objective function *F* be positive in hyperparallelepiped Π. Let the sequence ς(k) be defined as follows:

$$\varsigma(k) = (1/k)^{1/2n} .$$

Then for any point z in Π and any fixed positive real number δ, in the sequence of agents generated by the SoFAM algorithm, with probability one there exists an element belonging to the cubic δ-neighborhood of z.

Convergence of algorithm. Theorem 2



Let the objective function F be continuous, positive and have a unique global maximum point in Π. Let condition

$$\varsigma(k) = (1/k)^{1/2n}$$

be satisfied. Then for any fixed positive real number δ, the probability of choosing the agent at the k-th step of the SoFAM algorithm from the cubic δ-neighborhood of the global maximum point tends to one with probability one when $k \to \infty$.

Thank you for your attention

kuzenkov_o@mail.ru